

# Text Generation with LSTM in PyTorch

by [Adrian Tam](#) on [April 8, 2023](#) in [Deep Learning with PyTorch](#)  5

[Share](#) [Share](#) [Post](#)

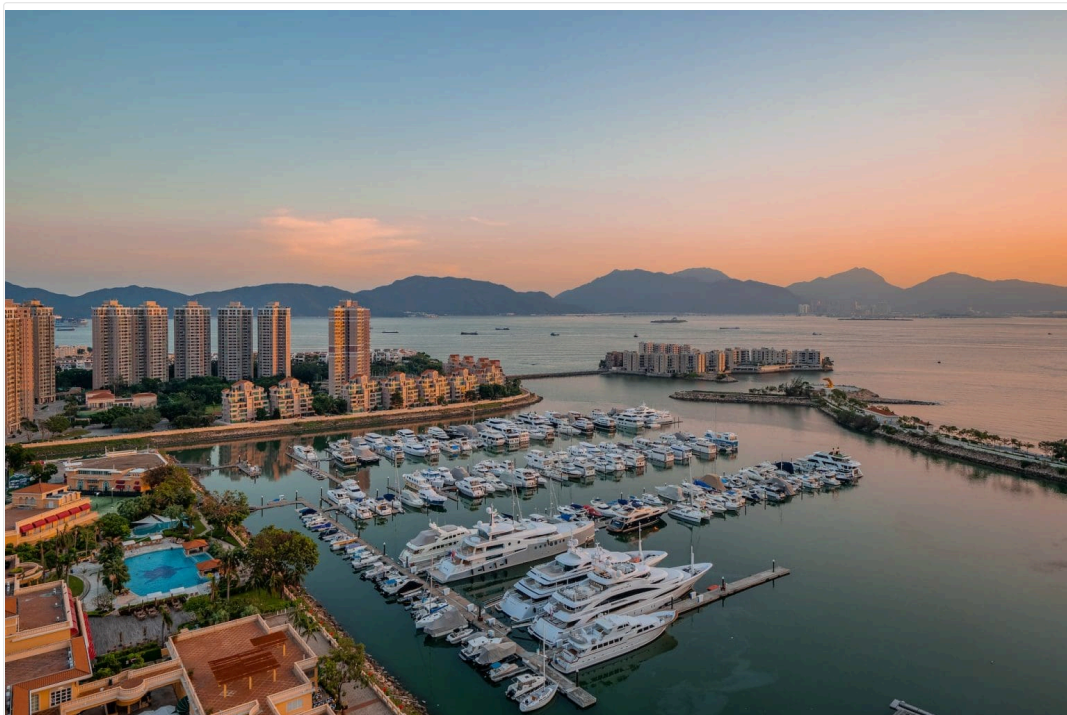
Recurrent neural network can be used for time series prediction. In which, a regression neural network is created. It can also be used as generative model, which usually is a classification neural network model. A generative model is to learn certain pattern from data, such that when it is presented with some prompt, it can create a complete output that in the same style as the learned pattern.

In this post, you will discover how to build a generative model for text using LSTM recurrent neural networks in PyTorch. After finishing this post, you will know:

- Where to download a free corpus of text that you can use to train text generative models
- How to frame the problem of text sequences to a recurrent neural network generative model
- How to develop an LSTM to generate plausible text sequences for a given problem

**Kick-start your project** with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Text Generation with LSTM in PyTorch  
Photo by [Egor Lyfar](#). Some rights reserved.

## Overview

This post is divided into six parts; they are:

- What is a Generative Model
- Getting Text Data

- A Small LSTM Network to Predict the Next Character
- Generating Text with an LSTM Model
- Using a Larger LSTM Network
- Faster Training with GPU

## What is a Generative Model

Generative model is indeed, just another machine learning model that happened to be able to create new things. Generative Adversarial Network (GAN) is a class of its own. Transformer models that uses attention mechanism is also found to be useful to generate text passages.

It is just a machine learning model because the model has been trained with existing data, so that it learned something from it. Depends on how to train it, they can work vastly different. In this post, a character-based generative model is created. What it means is to train a model that take a sequence of characters (alphabets and punctuations) as input and the immediate next character as the target. As long as it can predict what is the next character given what are preceding, you can run the model in a loop to generate a long piece of text.

This model is probably the simplest one. However, human language is complex. You shouldn't expect it can produce very high quality output. Even so, you need a lot of data and train the model for a long time before you can see sensible results.

### Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

## Getting Text Data

Obtaining high quality data is important for a successful generative model. Fortunately, many of the classical texts are no longer protected under copyright. This means you can download all the text for these books for free and use them in experiments, like creating generative models. Perhaps the best place to get access to free books that are no longer protected by copyright is Project Gutenberg.

In this post, you will use a favorite book from childhood as the dataset, Alice's Adventures in Wonderland by Lewis Carroll:

- <https://www.gutenberg.org/ebooks/11>

Your model will learn the dependencies between characters and the conditional probabilities of characters in sequences so that you can, in turn, generate wholly new and original sequences of characters. This post is a lot of fun, and repeating these experiments with other books from Project Gutenberg is recommended. These experiments are not limited to text; you can also experiment with other ASCII data, such as computer source code, marked-up documents in LATEX, HTML or Markdown, and more.

You can download the complete text in ASCII format (Plaintext UTF-8) for this book for free and place it in your working directory with the filename `wonderland.txt`. Now, you need to prepare the dataset ready for modeling. Project Gutenberg adds a standard header and footer to each book, which is not part of the original text. Open the file in a text editor and delete the header and footer. The header is obvious and ends with the text:

```
1 *** START OF THIS PROJECT GUTENBERG EBOOK ALICE'S ADVENTURES IN WONDERLAND ***
```

The footer is all the text after the line of text that says:

```
1 THE END
```

You should be left with a text file that has about 3,400 lines of text.

## A Small LSTM Network to Predict the Next Character

First, you need to do some preprocessing on the data before you can build a model. Neural network models can only work with numbers, not text. Therefore you need to transform the characters into numbers. To make the problem simpler, you also want to transform all uppercase letters into lowercase.

In below, you open the text file, transform all letters into lowercase, and create a Python dict `char_to_int` to map characters into distinct integers. For example, the list of unique sorted lowercase characters in the book is as follows:

```
1 ['\n', '\r', '!', '"', '#', '$', '%', '&', "'", '(', ')', '*', '+', ',', '-', '.', ':', ';', '?', '[', ']',
2 '\t', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p',
3 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', '\xbb', '\xbf', '\xef']
```

Since this problem is character-based, the “vocabulary” are the distinct characters ever used in the text.

```
1 import numpy as np
2
3 # load ascii text and covert to lowercase
4 filename = "wonderland.txt"
5 raw_text = open(filename, 'r', encoding='utf-8').read()
6 raw_text = raw_text.lower()
7
8 # create mapping of unique chars to integers
9 chars = sorted(list(set(raw_text)))
10 char_to_int = dict((c, i) for i, c in enumerate(chars))
11
12 # summarize the loaded data
13 n_chars = len(raw_text)
14 n_vocab = len(chars)
15 print("Total Characters: ", n_chars)
16 print("Total Vocab: ", n_vocab)
```

This should print:

```
1 Total Characters: 144574
2 Total Vocab: 50
```

You can see the book has just under 150,000 characters, and when converted to lowercase, there are only 50 distinct characters in the vocabulary for the network to learn — much more than the 26 in the alphabet.

Next, you need to separate the text into inputs and targets. A window of 100 character is used here. That is, with character 1 to 100 as input, your model is going to predict for character 101. Should a window of 5 be used, the word “chapter” will become two data samples:

```
1 chapt -> e
2 hapte -> r
```

In a long text such as this one, a myriad of windows can be created and this produced a dataset of a lot of samples:

```
1 # prepare the dataset of input to output pairs encoded as integers
2 seq_length = 100
3 dataX = []
4 dataY = []
5 for i in range(0, n_chars - seq_length, 1):
6     seq_in = raw_text[i:i + seq_length]
7     seq_out = raw_text[i + seq_length]
8     dataX.append([char_to_int[char] for char in seq_in])
9     dataY.append(char_to_int[seq_out])
10 n_patterns = len(dataX)
11 print("Total Patterns: ", n_patterns)
```

Running the above, you can see a total of 144,474 samples are created. Each sample is now in the form of integers, transformed using the mapping `char_to_int`. However, a PyTorch model would prefer to see the data in floating point tensors. Hence you should convert these into PyTorch tensors. LSTM layer is going to be used in the model, thus the input tensor should be of dimension (sample, time steps, features). To help training, it is also a good idea to normalize the input to 0 to 1. Hence you have the following:

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 # reshape X to be [samples, time steps, features]
6 X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
7 X = X / float(n_vocab)
8 y = torch.tensor(dataY)
9 print(X.shape, y.shape)
```

You can now define your LSTM model. Here, you define a single hidden LSTM layer with 256 hidden units. The input is single feature (i.e., one integer for one character). A dropout layer with probability 0.2 is added after the LSTM layer. The output of LSTM layer is a tuple, which the first element is the hidden states from the LSTM cell for each of the time step. It is a history of how the hidden state evolved as the LSTM cell accepts each time step of input. Presumably, the last hidden state contained the most information, hence only the last hidden state is pass on to the output layer. The output layer is a fully-connected layer to produce logits for the 50 vocabularies. The logits can be converted into probability-like prediction using a softmax function.

```
1 import torch.nn as nn
2 import torch.optim as optim
3 import torch.utils.data as data
4
5 class CharModel(nn.Module):
6     def __init__(self):
7         super().__init__()
8         self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=1, batch_first=True)
9         self.dropout = nn.Dropout(0.2)
10        self.linear = nn.Linear(256, n_vocab)
11
12    def forward(self, x):
13        x, _ = self.lstm(x)
14        # take only the last output
15        x = x[:, -1, :]
```

```

15     # produce output
16     x = self.linear(self.dropout(x))
17     return x

```

This is a model for single character classification of 50 classes. Therefore cross entropy loss should be used. It is optimized using Adam optimizer. The training loop is as follows. For simplicity, no test set has created, but the model is evaluated with the training set once again at the end of each epoch to keep track on the progress.

This program can run for a long time, especially on CPU! In order to preserve the fruit of work, the best model ever found is saved for future reuse.

```

1  n_epochs = 40
2  batch_size = 128
3  model = CharModel()
4
5  optimizer = optim.Adam(model.parameters())
6  loss_fn = nn.CrossEntropyLoss(reduction="sum")
7  loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)
8
9  best_model = None
10 best_loss = np.inf
11 for epoch in range(n_epochs):
12     model.train()
13     for X_batch, y_batch in loader:
14         y_pred = model(X_batch)
15         loss = loss_fn(y_pred, y_batch)
16         optimizer.zero_grad()
17         loss.backward()
18         optimizer.step()
19     # Validation
20     model.eval()
21     loss = 0
22     with torch.no_grad():
23         for X_batch, y_batch in loader:
24             y_pred = model(X_batch)
25             loss += loss_fn(y_pred, y_batch)
26         if loss < best_loss:
27             best_loss = loss
28             best_model = model.state_dict()
29         print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))
30
31 torch.save([best_model, char_to_dict], "single-char.pth")

```

Running the above may produce the following:

```

1  ...
2  Epoch 35: Cross-entropy: 245745.2500
3  Epoch 36: Cross-entropy: 243908.7031
4  Epoch 37: Cross-entropy: 238833.5000
5  Epoch 38: Cross-entropy: 239069.0000
6  Epoch 39: Cross-entropy: 234176.2812

```

The cross entropy almost always decreasing in each epoch. This means probably the model is not fully converged and you can train it for more epochs. Upon the training loop completed, you should have the file `single-char.pth` created to contain the best model weight ever found, as well as the character-to-integer mapping used by this model.

For completeness, below is tying everything above into one script:

```

1  import numpy as np
2  import torch
3  import torch.nn as nn
4  import torch.optim as optim
5  import torch.utils.data as data
6
7  # load ascii text and covert to lowercase
8  filename = "wonderland.txt"
9  raw_text = open(filename, 'r', encoding='utf-8').read()
10 raw_text = raw_text.lower()
11
12 # create mapping of unique chars to integers
13 chars = sorted(list(set(raw_text)))
14 char_to_int = dict((c, i) for i, c in enumerate(chars))
15
16 # summarize the loaded data
17 n_chars = len(raw_text)
18 n_vocab = len(chars)
19 print("Total Characters: ", n_chars)
20 print("Total Vocab: ", n_vocab)
21
22 # prepare the dataset of input to output pairs encoded as integers
23 seq_length = 100
24 dataX = []
25 dataY = []
26 for i in range(0, n_chars - seq_length, 1):
27     seq_in = raw_text[i:i + seq_length]
28     seq_out = raw_text[i + seq_length]
29     dataX.append([char_to_int[char] for char in seq_in])
30     dataY.append(char_to_int[seq_out])
31 n_patterns = len(dataX)
32 print("Total Patterns: ", n_patterns)
33
34 # reshape X to be [samples, time steps, features]

```

```

35 X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
36 X = X / float(n_vocab)
37 y = torch.tensor(dataY)
38
39 class CharModel(nn.Module):
40     def __init__(self):
41         super().__init__()
42         self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=1, batch_first=True)
43         self.dropout = nn.Dropout(0.2)
44         self.linear = nn.Linear(256, n_vocab)
45     def forward(self, x):
46         x, _ = self.lstm(x)
47         # take only the last output
48         x = x[:, -1, :]
49         # produce output
50         x = self.linear(self.dropout(x))
51         return x
52
53 n_epochs = 40
54 batch_size = 128
55 model = CharModel()
56
57 optimizer = optim.Adam(model.parameters())
58 loss_fn = nn.CrossEntropyLoss(reduction="sum")
59 loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)
60
61 best_model = None
62 best_loss = np.inf
63 for epoch in range(n_epochs):
64     model.train()
65     for X_batch, y_batch in loader:
66         y_pred = model(X_batch)
67         loss = loss_fn(y_pred, y_batch)
68         optimizer.zero_grad()
69         loss.backward()
70         optimizer.step()
71     # Validation
72     model.eval()
73     loss = 0
74     with torch.no_grad():
75         for X_batch, y_batch in loader:
76             y_pred = model(X_batch)
77             loss += loss_fn(y_pred, y_batch)
78         if loss < best_loss:
79             best_loss = loss
80             best_model = model.state_dict()
81     print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))
82
83 torch.save([best_model, char_to_int], "single-char.pth")

```

## Generating Text with an LSTM Model

Given the model is well trained, generating text using the trained LSTM network is relatively straightforward. Firstly, you need to recreate the network and load the trained model weight from the saved checkpoint. Then you need to create some prompt for the model to start on. The prompt can be anything that the model can understand. It is a seed sequence to be given to the model to obtain one generated character. Then, the generated character is added to the end of this sequence, and trim off the first character to maintain the consistent length. This process is repeated for as long as you want to predict new characters (e.g., a sequence of 1,000 characters in length). You can pick a random input pattern as your seed sequence, then print generated characters as you generate them.

A simple way to generate prompt is to pick a random sample from the original dataset, e.g., with the `raw_text` obtained in the previous section, a prompt can be created as:

```

1 seq_length = 100
2 start = np.random.randint(0, len(raw_text)-seq_length)
3 prompt = raw_text[start:start+seq_length]

```

But you should be reminded that you need to transform it since this prompt is a string while the model expects a vector of integers.

The entire code is merely as follows:

```

1 import numpy as np
2 import torch
3 import torch.nn as nn
4
5 best_model, char_to_int = torch.load("single-char.pth")
6 n_vocab = len(char_to_int)
7 int_to_char = dict((i, c) for c, i in char_to_int.items())
8
9 # reload the model
10 class CharModel(nn.Module):
11     def __init__(self):
12         super().__init__()
13         self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=1, batch_first=True)
14         self.dropout = nn.Dropout(0.2)
15         self.linear = nn.Linear(256, n_vocab)
16     def forward(self, x):
17         x, _ = self.lstm(x)
18         # take only the last output
19         x = x[:, -1, :]

```

```

20     # produce output
21     x = self.linear(self.dropout(x))
22     return x
23 model = CharModel()
24 model.load_state_dict(best_model)
25
26 # randomly generate a prompt
27 filename = 'wonderland.txt'
28 seq_length = 100
29 raw_text = open(filename, 'r', encoding='utf-8').read()
30 raw_text = raw_text.lower()
31 start = np.random.randint(0, len(raw_text)-seq_length)
32 prompt = raw_text[start:start+seq_length]
33 pattern = [char_to_int[c] for c in prompt]
34
35 model.eval()
36 print('Prompt: "%s"' % prompt)
37 with torch.no_grad():
38     for i in range(1000):
39         # format input array of int into PyTorch tensor
40         x = np.reshape(pattern, (1, len(pattern), 1)) / float(n_vocab)
41         x = torch.tensor(x, dtype=torch.float32)
42         # generate logits as output from the model
43         prediction = model(x)
44         # convert logits into one character
45         index = int(prediction.argmax())
46         result = int_to_char[index]
47         print(result, end="")
48         # append the new character into the prompt for the next iteration
49         pattern.append(index)
50         pattern = pattern[1:]
51 print()
52 print("Done.")

```

Running this example first outputs the prompt used, then each character as it is generated. For example, below are the results from one run of this text generator. The prompt was:

```

1 Prompt: "nother rush at the stick, and tumbled head
2 over heels in its hurry to get hold of it; then alice, th"

```

The generated text was:

```

1 e was qot a litule soteet of thet was sh the thiee harden an the courd, and was tuitk a little toaee th thite ththe and said to the suher, and
2
3 "i don't know the ter ano _enend to mere," said the maccht ar a sore of great roaee. "ie you don't teink if thet soued to soeed to the boeie t
4 tou het bn the crur,
5 "h whsh you cen not," said the manch hare.
6
7 "wes, it aadi," said the manch hare.
8
9 "weat you tail to merer ae in an a gens if gre" "he were thing," said the maccht ar a sore of geeaghen asd tothe to the thieg harden an the ce
10 "h dan tor toe taie thing," said the manch hare.
11
12 "wes, it aadi," said the manch hare.
13
14 "weat you tail to merer ae in an a gens if gre" "he were thing," said the maccht ar a sore of geeaghen asd tothe to the thieg harden an t

```

Let's note some observations about the generated text.

- It can emit line breaks. The original text limited the line width to 80 characters and the generative model attempted to replicate this pattern
- The characters are separated into word-like groups, and some groups are actual English words (e.g., "the," "said," and "rabbit"), but many are not (e.g., "thite," "soteet," and "tha").
- Some of the words in sequence make sense (e.g., "i don't know the"), but many do not (e.g., "he were thing").

The fact that this character-based model of the book produces output like this is very impressive. It gives you a sense of the learning capabilities of LSTM networks. However, the results are not perfect. In the next section, you will look at improving the quality of results by developing a much larger LSTM network.

## Using a Larger LSTM Network

Recall that LSTM is a recurrent neural network. It takes a sequence as input, which in each step of the sequence, the input is mixed with its internal states to produce an output. Hence the output from LSTM is also a sequence. In the above, the output from the last time step is taken for further processing in the neural network but those from earlier steps are discarded. However, it is not necessarily the case. You can treat the sequence output from one LSTM layer as input to another LSTM layer. Then, you are building a larger network.

Similar to convolutional neural networks, a stacked LSTM network is supposed to have the earlier LSTM layers to learn low level features while the later LSTM layers to learn the high level features. It may not be always useful but you can try it out to see whether the model can produce a better result.

In PyTorch, making a stacked LSTM layer is easy. Let's modify the above model into the following:

```

1 class CharModel(nn.Module):
2     def __init__(self):
3         super().__init__()

```

```

4     self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=2, batch_first=True, dropout=0.2)
5     self.dropout = nn.Dropout(0.2)
6     self.linear = nn.Linear(256, n_vocab)
7     def forward(self, x):
8         x, _ = self.lstm(x)
9         # take only the last output
10        x = x[:, -1, :]
11        # produce output
12        x = self.linear(self.dropout(x))
13        return x

```

The only change is on the parameter to `nn.LSTM()`: you set `num_layers=2` instead of 1 to add another LSTM layer. But between the two LSTM layers, you also added a dropout layer through the parameter `dropout=0.2`. Replacing this model with the previous is all the change you need to make. Rerun the training you should see the below:

```

1 ...
2 Epoch 34: Cross-entropy: 203763.0312
3 Epoch 35: Cross-entropy: 204002.5938
4 Epoch 36: Cross-entropy: 210636.5625
5 Epoch 37: Cross-entropy: 199619.6875
6 Epoch 38: Cross-entropy: 199240.2969
7 Epoch 39: Cross-entropy: 196966.1250

```

You should see the the cross entropy here is lower than that in the previous section. This means this model is performing better. In fact, with this model, you can see the generated text looks more sensible:

```

1 Prompt: "ll
2 say that 'i see what i eat' is the same thing as 'i eat what i see!'"
3
4 "you might just as well sa"
5 y it to sea," she katter said to the jury. and the thoe hardeners vhine she was seady to alic the was a long tay of the sooe of the court,
6 "well you see what you see, the mookee of the soog of the season of the shase of the court!"
7
8 "i don't know the rame thing is it?" said the caterpillar.
9
10 "the cormous was it makes he it was it taie the reason of the shall bbout it, you know."
11
12 "i don't know the rame thing i can't gelp the sea," the hatter went on, "i don't know the peally was in the shall sereat it would be a teally
13 the mookee of the court "
14
15 "i don't know the rame thing is it?" said the caterpillar.
16
17 "the cormous was it makes he it was it taie the reason of the shall bbout it, you know."
18
19 "i don't know the rame thing i can't gelp the sea," the hatter went on, "i don't know the peally was in the shall sereat it would be a teally
20 the mookee of the court "
21
22 "i don't know the rame thing is it?" said the caterpillar.
23
24 "the
25 Done.

```

Not only words are spelled correctly, the text is also more English-like. Since the cross-entropy loss is still decreasing as you trained the model, you can assume the model is not converged yet. You can expect to make the model better if you increased the training epoch.

For completeness, below is the complete code for using this new model, including training and text generation.

```

1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 import torch.utils.data as data
6
7 # load ascii text and covert to lowercase
8 filename = "wonderland.txt"
9 raw_text = open(filename, 'r', encoding='utf-8').read()
10 raw_text = raw_text.lower()
11
12 # create mapping of unique chars to integers
13 chars = sorted(list(set(raw_text)))
14 char_to_int = dict((c, i) for i, c in enumerate(chars))
15
16 # summarize the loaded data
17 n_chars = len(raw_text)
18 n_vocab = len(chars)
19 print("Total Characters: ", n_chars)
20 print("Total Vocab: ", n_vocab)
21
22 # prepare the dataset of input to output pairs encoded as integers
23 seq_length = 100
24 dataX = []
25 dataY = []
26 for i in range(0, n_chars - seq_length, 1):
27     seq_in = raw_text[i:i + seq_length]
28     seq_out = raw_text[i + seq_length]
29     dataX.append([char_to_int[char] for char in seq_in])
30     dataY.append(char_to_int[seq_out])
31 n_patterns = len(dataX)
32 print("Total Patterns: ", n_patterns)
33
34 # reshape X to be [samples, time steps, features]
35 X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)

```



```

36 X = X / float(n_vocab)
37 y = torch.tensor(dataY)
38
39 class CharModel(nn.Module):
40     def __init__(self):
41         super().__init__()
42         self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=2, batch_first=True, dropout=0.2)
43         self.dropout = nn.Dropout(0.2)
44         self.linear = nn.Linear(256, n_vocab)
45     def forward(self, x):
46         x, _ = self.lstm(x)
47         # take only the last output
48         x = x[:, -1, :]
49         # produce output
50         x = self.linear(self.dropout(x))
51         return x
52
53 n_epochs = 40
54 batch_size = 128
55 model = CharModel()
56
57 optimizer = optim.Adam(model.parameters())
58 loss_fn = nn.CrossEntropyLoss(reduction="sum")
59 loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)
60
61 best_model = None
62 best_loss = np.inf
63 for epoch in range(n_epochs):
64     model.train()
65     for X_batch, y_batch in loader:
66         y_pred = model(X_batch)
67         loss = loss_fn(y_pred, y_batch)
68         optimizer.zero_grad()
69         loss.backward()
70         optimizer.step()
71     # Validation
72     model.eval()
73     loss = 0
74     with torch.no_grad():
75         for X_batch, y_batch in loader:
76             y_pred = model(X_batch)
77             loss += loss_fn(y_pred, y_batch)
78         if loss < best_loss:
79             best_loss = loss
80             best_model = model.state_dict()
81     print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))
82
83 torch.save([best_model, char_to_int], "single-char.pth")
84
85 # Generation using the trained model
86 best_model, char_to_int = torch.load("single-char.pth")
87 n_vocab = len(char_to_int)
88 int_to_char = dict((i, c) for c, i in char_to_int.items())
89 model.load_state_dict(best_model)
90
91 # randomly generate a prompt
92 filename = "wonderland.txt"
93 seq_length = 100
94 raw_text = open(filename, 'r', encoding='utf-8').read()
95 raw_text = raw_text.lower()
96 start = np.random.randint(0, len(raw_text)-seq_length)
97 prompt = raw_text[start:start+seq_length]
98 pattern = [char_to_int[c] for c in prompt]
99
100 model.eval()
101 print('Prompt: "%s"' % prompt)
102 with torch.no_grad():
103     for i in range(1000):
104         # format input array of int into PyTorch tensor
105         x = np.reshape(pattern, (1, len(pattern), 1)) / float(n_vocab)
106         x = torch.tensor(x, dtype=torch.float32)
107         # generate logits as output from the model
108         prediction = model(x)
109         # convert logits into one character
110         index = int(prediction.argmax())
111         result = int_to_char[index]
112         print(result, end="")
113         # append the new character into the prompt for the next iteration
114         pattern.append(index)
115         pattern = pattern[1:]
116 print()
117 print("Done.")

```

## Faster Training with GPU

Running programs from this post can be pathetically slow. Even if you have a GPU, you will not see immediate improvement. It is because the design of PyTorch, it may not use your GPU automatically. However, if you have a CUDA-capable GPU, you can improve the performance a lot by carefully moving the heavy computation away from your CPU.

A PyTorch model is a program of tensor calculation. The tensors can be stored in GPU or in CPU. Operation can be carried out as long as all the operators are in the same device. In this particular example, the model weight (i.e., those of the LSTM layers and the fully connected layer) can be moved to GPU. By doing so, the input should also be moved to the GPU before execution and the output will also be stored in the GPU unless you move it back.



In PyTorch, you can check if you have a CUDA-capable GPU using the following function:

```
1 torch.cuda.is_available()
```

It returns a boolean to indicate if you can use GPU, which in turn, depends on the hardware model you have, whether your OS has the appropriate library installed, and whether your PyTorch is compiled with corresponding GPU support. If everything works in concert, you can create a device and assign your model to it:

```
1 device = torch.device("cuda:0")
2 model.to(device)
```

If your model is running on CUDA device but your input tensor is not, you will see PyTorch complain about that and fail to proceed. To move your tensor to the CUDA device, you should run like the following:

```
1 y_pred = model(X_batch.to(device))
```

Which the `.to(device)` part will do the magic. But remember that `y_pred` produced above will also be on the CUDA device. Hence you should do the same when you run the loss function. Modifying the above program to make it capable to run on GPU will become the following:

```
1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 import torch.utils.data as data
6
7 # load ascii text and covert to lowercase
8 filename = "wonderland.txt"
9 raw_text = open(filename, 'r', encoding='utf-8').read()
10 raw_text = raw_text.lower()
11
12 # create mapping of unique chars to integers
13 chars = sorted(list(set(raw_text)))
14 char_to_int = dict((c, i) for i, c in enumerate(chars))
15
16 # summarize the loaded data
17 n_chars = len(raw_text)
18 n_vocab = len(chars)
19 print("Total Characters: ", n_chars)
20 print("Total Vocab: ", n_vocab)
21
22 # prepare the dataset of input to output pairs encoded as integers
23 seq_length = 100
24 dataX = []
25 dataY = []
26 for i in range(0, n_chars - seq_length, 1):
27     seq_in = raw_text[i:i + seq_length]
28     seq_out = raw_text[i + seq_length]
29     dataX.append([char_to_int[char] for char in seq_in])
30     dataY.append(char_to_int[seq_out])
31 n_patterns = len(dataX)
32 print("Total Patterns: ", n_patterns)
33
34 # reshape X to be [samples, time steps, features]
35 X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
36 X = X / float(n_vocab)
37 y = torch.tensor(dataY)
38
39 class CharModel(nn.Module):
40     def __init__(self):
41         super().__init__()
42         self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=2, batch_first=True, dropout=0.2)
43         self.dropout = nn.Dropout(0.2)
44         self.linear = nn.Linear(256, n_vocab)
45     def forward(self, x):
46         x, _ = self.lstm(x)
47         # take only the last output
48         x = x[:, -1, :]
49         # produce output
50         x = self.linear(self.dropout(x))
51         return x
52
53 n_epochs = 40
54 batch_size = 128
55 model = CharModel()
56 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
57 model.to(device)
58
59 optimizer = optim.Adam(model.parameters())
60 loss_fn = nn.CrossEntropyLoss(reduction="sum")
61 loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)
62
63 best_model = None
64 best_loss = np.inf
65 for epoch in range(n_epochs):
66     model.train()
67     for X_batch, y_batch in loader:
68         y_pred = model(X_batch.to(device))
69         loss = loss_fn(y_pred, y_batch.to(device))
70         optimizer.zero_grad()
71         loss.backward()
72         optimizer.step()
73     # Validation
```

```

74     model.eval()
75     loss = 0
76     with torch.no_grad():
77         for X_batch, y_batch in loader:
78             y_pred = model(X_batch.to(device))
79             loss += loss_fn(y_pred, y_batch.to(device))
80         if loss < best_loss:
81             best_loss = loss
82             best_model = model.state_dict()
83         print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))
84
85 torch.save([best_model, char_to_int], "single-char.pth")
86
87 # Generation using the trained model
88 best_model, char_to_int = torch.load("single-char.pth")
89 n_vocab = len(char_to_int)
90 int_to_char = dict((i, c) for c, i in char_to_int.items())
91 model.load_state_dict(best_model)
92
93 # randomly generate a prompt
94 filename = "wonderland.txt"
95 seq_length = 100
96 raw_text = open(filename, 'r', encoding='utf-8').read()
97 raw_text = raw_text.lower()
98 start = np.random.randint(0, len(raw_text)-seq_length)
99 prompt = raw_text[start:start+seq_length]
100 pattern = [char_to_int[c] for c in prompt]
101
102 model.eval()
103 print('Prompt: "%s"' % prompt)
104 with torch.no_grad():
105     for i in range(1000):
106         # format input array of int into PyTorch tensor
107         x = np.reshape(pattern, (1, len(pattern), 1)) / float(n_vocab)
108         x = torch.tensor(x, dtype=torch.float32)
109         # generate logits as output from the model
110         prediction = model(x.to(device))
111         # convert logits into one character
112         index = int(prediction.argmax())
113         result = int_to_char[index]
114         print(result, end="")
115         # append the new character into the prompt for the next iteration
116         pattern.append(index)
117         pattern = pattern[1:]
118     print()
119     print("Done.")

```

Compare to the code in the previous section, you should see they are essentially the same. Except the CUDA device is detected with the line:

```
1 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

which will be your GPU or fall back to CPU if no CUDA device is found. Afterward, `.to(device)` is added at several strategic location to move the computation to the GPU.

## Further Readings

This character text model is a popular way of generating text using recurrent neural networks. Below are some more resources and tutorials on the topic if you are interested in going deeper.

### Articles

- Andrej Karpathy. The Unreasonable Effectiveness of Recurrent Neural Networks. May 2015.
- Lars Eidnes. Auto-Generating Clickbait With Recurrent Neural Networks. 2015.
- PyTorch tutorial. Sequence Models and Long Short-Term Memory Networks

### Papers

- Ilya Sutskever, James Martens, and Geoffrey Hinton. "Generating Text with Recurrent Neural Networks". In: Proceedings of the 28th International Conference on Machine Learning. Bellevue, WA, USA, 2011.

### APIs

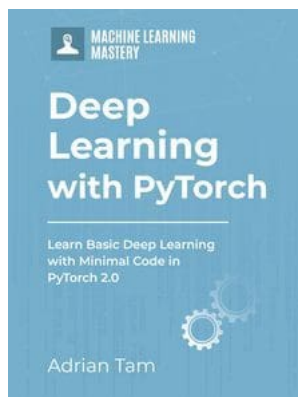
- nn.LSTM in PyTorch documentation

## Summary

In this post, you discovered how you can develop an LSTM recurrent neural network for text generation in PyTorch. After completing this post, you know:

- How to find text for classical books for free as dataset for your machine learning model
- How to train an LSTM network for text sequences
- How to use a LSTM network to generate text sequencesHow to optimize deep learning training in PyTorch using CUDA devices

## Get Started on Deep Learning with PyTorch!



### Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:  
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

### Kick-start your deep learning journey with hands-on exercises

SEE WHAT'S INSIDE

Share Share Post

### More On This Topic



**Text Generation With LSTM  
Recurrent Neural Networks...**



**LSTM for Time Series  
Prediction in PyTorch**



**Understanding Text  
Generation Parameters in  
Transformers**



**Text Embedding Generation  
with Transformers**



**Text Generation with GPT-2  
Model**



**Auto-Completion Style Text  
Generation with GPT-2 Model**



#### About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.  
View all posts by Adrian Tam →

< LSTM for Time Series Prediction in PyTorch

Building an Image Classifier with a Single-Layer Neural Network in PyTorch >

## 5 Responses to *Text Generation with LSTM in PyTorch*



**Zineb** April 4, 2023 at 9:27 pm #

REPLY ↩

Thanks a lot for this wonderful article

`torch.save([best_model, char_to_dict], "single-char.pth")` should be `torch.save([best_model, char_to_int], "single-char.pth")`



**Emmett Polhemus** March 20, 2024 at 5:05 am #

REPLY ↩

I am having trouble were the algorithm does not print the final output and instead stops.



**James Carmichael** March 20, 2024 at 8:49 am #

REPLY ↩

Hi Emmett...Please provide any specific error messages you may be encountering. That will better enable us to guide you.



**Emmett polhemus** March 21, 2024 at 10:10 pm #

REPLY ↩

I have gotten it working the error had to do with the signle-char.path file I was using a .txt. Also, do you need to train it every time you want a prompt?



**shadow** April 8, 2024 at 10:41 am #

REPLY ↩

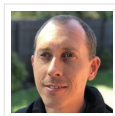
Very nice

## Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT



**Welcome!**

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

Never miss a tutorial:



### Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Develop Your First Neural Network with PyTorch, Step by Step

### Loving the Tutorials?

The [Deep Learning with PyTorch](#) EBook  
is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved